第 9 章推導解答

DS09-01 • Leftist rank 與 meld

H_1 根 3 的左右 rank 均 1，根 rank=2。H_2 根 4 左 rank=1、右=0，根 rank=1。meld 保留 3，遞迴合 9 與 4，保留 4，將 9 接為其右子，兩子 8、9 均 rank=1，故 4.rank=2。回到 3，左子 7.rank=1 小於右子 4.rank=2，交換，得到左 4、右 7，3.rank=2。最小點數由 N(0)=0、N(r)≥1+2N(r−1) 得 2^r−1。鍵遞增的純左鏈每個非空點右 rank=0、左 rank≥0，全部 rank=1，仍符合 leftist，但高度 n−1。

DS09-02 • Binomial 進位與插入攤銷

5 的 degree 為 {0,2}，3 為 {0,1}。兩 B_0 合成 B_1，與原 B_1 合成 B_2，再與原 B_2 合成 B_3，剩 8 點一樹。專用 insert 加一根，再做 k 次 link，實際成本可界 a(k+1)。根數變化 Δt=1−k，選 Φ=a・t，可得攤銷成本至多 a(k+1)+a(1−k)=2a。若實作即使沒有進位也掃完整 O(log n) 根清單，勢能不會自動支付額外掃描；此插入攤銷證明要求遇到無同 degree 根即可停止，或另有能界定掃描總量的設計。

DS09-03 • Fibonacci cascading cuts 的作用

c 次切斷使 t 增 c；前 c−1 個已標記節點切成根後清標，m 至少減 c−1，但可能在停止祖先新標一點。因此 ΔΦ 至多 c−2(c−1)+2=4−c。實際操作每切有常數 a，將勢能整體乘足夠常數即可支付 O(c) 並留下 O(1) 攤銷成本。沒有 cascading cuts，某個仍留在父下的子節點可失去任意多後代，其 degree 可能遠低於連入時 degree−1，因此第 i 子仍至少 i−2 degree 的推導不成立，無法由此得到對數 degree 上界。

DS09-04 • Pairing heap 的操作與共同勢能分析

第一輪(8,3)得3含子8，(9,5)得5含子9，7保留。第二輪先link(5,7)，5的子為7,9；再link(3,5)，3的子為5,8。原5以下的7,9仍在，共四次link。

LCRS的s(v)計入右兄弟及其後代，不是原多叉子樹大小。Φ=Σ$\log_2$s(v)≥0，空堆為0；獨立堆分別計算，再相加。刪根後用保序forest的虛擬兄弟鏈分析：即使程序把根暫存陣列，也依分量順序解讀，而不把逐個清next的瞬間另算勢能。

相鄰A,B的原大小a,b≥1，右後綴R≥0。link前兩根二元大小a+b+R、b+R；之後勝根a+b+R、敗根a+b−1。其餘節點不變，兩種勝負均有δ=$\log_2$(a+b−1)−$\log_2$(b+R)。R>0時令A=a+b、T=A+R，則1+δ=$\log_2$[2(A−1)/(b+R)]≤$\log_2$(2A/R)≤2$\log_2$(T/R)，最後一步來自2AR≤$(A+R)^2$。

設N=n−1、q=⌊k/2⌋。第一輪相鄰兩對的R_j=T_(j+1)，所以能望遠鏡相消。若k=2q+1，末棵大小r≥1，所有R>0，q+ΔΦ₁≤2$\log_2$(N/r)≤2$\log_2$N。若k=2q，最後一對R=0、總大小A_q≥2；前q−1對有(q−1)+ΔΦ_前≤2$\log_2$(N/A_q)，最後δ≤$\log_2$A_q。故ΔΦ₁≤−q+1+2$\log_2$N。q=1時前和為0；不能對最後一對除以R=0。

第 9 章推導解答 （續）

第二輪每次右累積堆大小b、左棵大小a，R=0，δ≤log₂((a+b)/b)。下一次的b即本次a+b，故ΔΦ₂≤log₂(N/b₀)≤log₂N，b₀為最右分量大小；只有一分量時空和為0。移根使勢能少log₂n，因此k≥2時ΔΦ≤−q+3log₂n+1。若實際工作≤ck+d、k≤2q+1，固定Ψ=CΦ且C≥2c，攤銷≤(2c−C)q+3C log₂n+c+d+C=O(1+log n)。掃描及兩輪所有k−1次link均包含在ck+d中。k=0為刪唯一點，ΔΦ=0；k=1無link、ΔΦ=−log₂n，兩者實際O(1)。空刪除拒絕，暫存陣列空間O(k)。

五單點例第一輪精確比值1/4、1/2，第二輪2、4/3；移根比值1/6，整次Φ差=log₂(1/18)。這只是核對一般推導的數例，並非一般證明的替代。

meld的起點是兩個獨立堆，故非空大小a,b、n=a+b時ΔΦ=log₂[n(n−1)/(ab)]≤log₂n，因ab≥n−1；空操作數給0。insert是與單點meld，非空插入精確增加log₂n。兩者實際O(1)，但此Ψ均收O(1+log(n+1))攤銷。

降根鍵不改勢能。非根x切開後，x失去右兄弟後綴，原二元祖先只會失去可達點，其餘s不變，故切成兩獨立堆的ΔΦ≤0。再meld回去增加≤log₂n，得同一Ψ下O(1+log(n+1))攤銷。當下cut/link為O(1)須已持合法handle並維護parent、prev、next；只有next可能需線性找前驅。

從空堆集合開始Ψ₀=0、Ψ_末≥0，實際總和≤攤銷總和。最多存活N點，I,M,D,K,F次insert,meld,delete-min,decrease-key,find-min，得到O((I+M+D+K)(1+log₂(N+1))+F)，另建空堆每次O(1)。此證明未給insert/meld常數攤銷，故不可改寫成O(I+M+(D+K)log N+F)。

DS09-05 • 工作負載選型

若先建立含 V 點的 indexed binary heap，decrease-key 已有 handle 且每次 O(log V)，delete-min O(log V)，總 O((V+E)log V)，建堆可另加 O(V)。Fibonacci 由空堆插入 V 次、V 次 delete-min、E 次 decrease-key，總 O(V log V+E) 攤銷。這些上界不包括任意大小鍵比較、cache miss、配置及實作常數。Fibonacci 的多指標與切斷可能增加實機成本，必須用相同資料與工作負載量測才能比較，不能從漸近式虛構速度倍數。

DS09-06 • 直接刪除、handle生命週期與共同勢能

起始n32，根v26(key−11,d0)、v18(key−10,d0)、v1(key1,d5)，marked={v17,v25}，Φ/A=7。v29的父是v25，子環為v31(d1，其子v32)、v30(d0)。先cut(v29,v25)，v25.degree從2降1；v25已標記，cut(v25,v17)並清其mark，v17.degree從3降2；v17也已標記，cut(v17,v1)並清mark，v1.degree從5降4。v1是根，停止。三次cut後t6,m0，Φ/A=6。

第 9 章推導解答 （續）

提升v31、v30時，各從v29子環摘下、清parent/mark後接根環，v29.degree降至0；t8。摘除v29並使h29失效，n31、t7。快照v30d0,v31d1,v17d2,v25d1,v26d0,v18d0,v1d4依序處理：v25贏v31升d2；v17贏v25升d3；v26(key−11)贏v30升d1。最後A0=v18,A1=v26,A3=v17,A4=v1，t4,m0、Φ/A=4，min=v26且key−11。

全程沒有交換item或替換節點身分。h31、h30即使在提升後又變子節點，仍指原物件；所有未刪handle同理。h29再使用必拒絕；等鍵節點可能有不同payload與外部參照，刪除任一等鍵不能實作指定handle刪除。外堆handle在解參照前拒絕，不能拿它修改此堆的環。只cut v29不cascade會讓已標記的非根v25再次失子仍留在父底下，違反最多失一子的degree論證。

切斷階段k次cut有Δ(t+2m)≤4−k，以A支付ak+b而得O(1)攤銷。提昇／刪根／整併階段以切後t′根為起點，實際O(t′+d+D+1)、最後至多D+1根且mark不增，勢能下降抵消t′，剩O(d+D+1)=O(1+log(n+1))。兩段共用同一A，勢能差相加才是整次erase；此例ΔΦ/A=−3，但一般界不依賴這個例子的數值。目標已是根就省略cut/cascade；最後一點的子環空，摘除後n0、roots=min=null；空delete-min或失效handle先拒絕。單次仍可線性，不能把攤銷界改稱最壞對數。

DS09-07

> **三次 link。** 7 勝 9，得 7:[9]、degree=1，總根數 3。3 勝 7，得 3:[7,5] 且 7:[9]、degree=2，總根數 2。2 勝 3，得 2:[3,4,6]，3:[7,5]，7:[9]，4:[8]，degree(2)=3，總根數 1。節點集合始終為 {2,3,4,5,6,7,8,9}；每次移動的是整棵敗根子樹。
>
> **刪根。** 移除 2，原子串列根 [3,4,6] 的 degree 為 [2,1,0]。反轉後根 [6,4,3] 對應 B0={6}、B1={4,8}、B2={3,7,5,9}；共 1+2+4=7 點，min=3，三個新根 parent=null。本例不需新 link。根依 degree 排列，不能改成鍵值順序 [3,4,6]。
>
> **7+3 的進位。** degree 0 有 1+1+0=2，保留 0、進位 1；degree 1 有 1+1+1=3，保留 1、進位 1；degree 2 有 1+0+1=2，保留 0、進位 1；degree 3 有 0+0+1=1，保留 1、進位 0。故最後 B1、B3，共 10 點。三棵 B1 共 6 點，不能形成節點數為 2 的冪的單棵 Bk；必須保留一棵、link 另外兩棵。
>
> **結構前提。** Binomial heap 保留精確的二項樹形；Fibonacci heap 的節點可因 cut 失去子樹。相同 degree 的 link 只保證父 degree 增一及 heap order，並不恢復所有二項子樹大小。Fibonacci 的對數 degree 界須用失子限制與 Fibonacci 下界另證；「恰有 2^k 點」不能套用。

第 9 章推導解答 （續）

DS09-08 • 完整回程與操作結果

1. 合併。 A 根的左右 rank 為 1、1，r(A)=2；B 根的左右 rank 為 1、0，r(B)=1。

步驟	操作及結果
meld(3,4)	3<4，保留 3 與左子樹 6，遞迴 meld(10,4)
meld(10,4)	4<10，保留 4 與左子樹 8，遞迴 meld(null,10)
基例	回傳單點 10，rank=1，無鍵比較
回到 4	左 8、右 10 的 rank 皆 1，不交換，r(4)=2
回到 3	左 6 的 rank=1，右 4 的 rank=2；交換，r(3)=2

結果為 `3:(4,6)`、`4:(8,10)`、`6:(9,null)`，共 6 點，右鏈 3→6。

2. 刪根。 meld(4,6) 保留 4，遞迴 meld(10,6) 保留 6，再遞迴 meld(null,10)。回到 6 得左右根 9、10，rank 皆 1，r(6)=2。回到 4，左 8 的 rank=1、右 6 的 rank=2，交換。結果為 `4:(6,8)`、`6:(9,10)`，5 點、根 rank=2、min=4。

插入 2 時，meld(2,4) 保留新根 2，基例 meld(null,4) 回傳原堆。新根左空、右 rank=2，交換後為 `2:(4,null)`，下方保留 `4:(6,8)`、`6:(9,10)`。6 點、根 rank=1、min=2。

3. 成本。 比較兩次，上界 2+1−1=2，恰好達界。合法左鏈可有 n 點、height=n−1、rank=1；meld 不走整條左鏈。誤將高度當成 rank，就無法得到對數最壞界。

4. 前提。 三個根的 npl 依序為 1、0、1。Heap order 本來就要求父鍵≤子鍵，允許相等鍵**不需改動**；a.key>b.key 時才交換根，等鍵保留 a。互不重疊是元素分割歸納的前提；meld(H,H) 可讓同一節點被重複接入甚至成環，不是複製元素。

第 9 章推導解答 （續）

原創驗算程式附錄: leftist.cpp

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <random>
#include <stdexcept>
#include <utility>
#include <vector>

struct Node {
	explicit Node(int item) : key(item) {}
	int key;
	int rank = 1;
	std::unique_ptr<Node> left;
	std::unique_ptr<Node> right;
};

using Heap = std::unique_ptr<Node>;

int rank(const Heap& heap) {
	return heap ? heap->rank : 0;
}

Heap meld(Heap first, Heap second) {
	if (!first) {
		return second;
	}
	if (!second) {
		return first;
	}
	if (first->key > second->key) {
		std::swap(first, second);
	}
	first->right = meld(std::move(first->right), std::move(second));
	if (rank(first->left) < rank(first->right)) {
		std::swap(first->left, first->right);
	}
	first->rank = rank(first->right) + 1;
	return first;
}

void insert(Heap& heap, int key) {
	heap = meld(std::move(heap), std::make_unique<Node>(key));
}

int remove_min(Heap& heap) {
	if (!heap) {
		throw std::out_of_range("empty priority queue");
	}
	const int result = heap->key;
	auto old_root = std::move(heap);
	heap = meld(std::move(old_root->left), std::move(old_root->right));
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章推導解答 （續）

```
	return result;
}

std::size_t validate(const Heap& heap) {
	if (!heap) {
		return 0;
	}
	if (rank(heap->left) < rank(heap->right) || heap->rank != rank(heap->right) + 1 ||
		(heap->left && heap->left->key < heap->key) ||
		(heap->right && heap->right->key < heap->key)) {
		throw std::runtime_error("leftist invariant was violated");
	}
	return 1 + validate(heap->left) + validate(heap->right);
}

int main() {
	Heap first;
	Heap second;
	std::vector<int> expected;
	std::mt19937 random(20261024);
	for (int i = 0; i < 240; ++i) {
		const int value = static_cast<int>(random() % 101) - 50;
		insert(i % 2 == 0 ? first : second, value);
		expected.push_back(value);
		validate(first);
		validate(second);
	}
	auto combined = meld(std::move(first), std::move(second));
	if (first || second || validate(combined) != expected.size()) {
		throw std::runtime_error("destructive meld ownership or size differs");
	}
	std::sort(expected.begin(), expected.end());
	for (const int value : expected) {
		if (remove_min(combined) != value) {
			throw std::runtime_error("priority queue output order differs");
		}
		validate(combined);
	}
	bool rejected = false;
	try {
		remove_min(combined);
	} catch (const std::out_of_range&) {
		rejected = true;
	}
	if (!rejected) {
		throw std::runtime_error("empty removal was accepted");
	}
	std::cout << "PASS leftist: 240 inserts, meld ownership, invariant checks, sorted
drain, empty\n";
}
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 9 章推導解答 （續）

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study